

ソフトウェアデザイン 2026

学生向け演習

第2～15回

情報システム学科 3年生 選択2単位

授業で案内する提出先・期限に従って使用する。

演習のフォルダ構成は本教材の推奨例。

第2回 Pythonの基礎

到達目標と準備

値・反復・関数を使い、入力の契約をテストで説明する。Python 3.12以上を確認し、リポジトリのルートでコマンドを実行する。第1回の環境準備を済ませておく。

A 予測と追跡

`prices = [200, 120, 80]` をforで合計する。各反復のtotalを紙かMarkdownに記録する。空、重複、順序変更の3例について、実行前の予測と実行結果を比較する。

B 注文集計

`submission/02/summary.py` に `summary(orders) -> int` を実装する。

- `orders`は辞書のリスト。各辞書に `name` (文字列)、`price`、`quantity` を持つ。
- `price`と`quantity`は0以上の整数。`bool`は受け付けない。
- 戻り値は円単位の整数。注文なしは0。
- キー不足・不正な型・負数には理由の分かる `ValueError` を送出する。
- 入力のリストや辞書を変更しない。

受入例：

```
orders = [  
    {"name": "ノート", "price": 200, "quantity": 2},  
    {"name": "ペン", "price": 120, "quantity": 3},  
]  
assert summary(orders) == 760  
assert summary([]) == 0
```

計算関数を作ってから、ファイル末尾の `if __name__ == "__main__":` でデモを表示する。解説例は `examples/02-basics/order_summary.py`。最初は自分で仕様を例にしてから参照する。

C 検証

submission/02/test_summary.py にunittestを作る。通常、空、個数0、負数、文字列、bool、キー不足、入力非変更を試す。期待値は実装と独立に手で求める。

```
python3 submission/02/summary.py
python3 -m unittest discover -s submission/02 -v
```

失敗を1つ記録し、原因と修正を説明する。assert文は最初の確認に使えるが、ユーザー入力の検証はifとraiseで行う。

提出するもの

summary.py、test_summary.py、README.md（実行方法と仕様）、reflection.md（予測と結果、失敗と修正、AIの利用）をひとまとめにする。提出先と期限は授業内の案内に従う。このフォルダ構成は教材の推奨例。

AIを使った場合は、依頼内容、採用・修正した提案、反例またはテスト、最終的に自分で説明できる部分を記録する。利用していない場合はその旨を書く。実在の注文や個人情報を入力に使わない。

確認と発展

range(3)の回数、printとreturnの違い、空の合計0の理由、型注釈と入力検証の違いをコードで説明する。発展は商品ごとの小計を別関数に分け、同じテストを保つ。

第3回 AI時代の学習と開発

到達目標

生成AIを学習と開発に使い、採用の判断を反例・実行結果・説明で支える。Python 3.12以上、エディタ、第2回の集計コードを用意する。サービス固有の有料契約は教材の必須条件ではない。

A 学習の依頼

説明しづらいPythonの概念を1つ選ぶ。自分の予測・疑問を書き、AIに小さな例や質問を依頼する。回答と異なる入力を自分で作り、手元で実行する。自分の説明がどのように変わったか200字程度で残す。

B 二分探索の診断

examples/03-ai-workflow/binary_search_buggy.pyは検証教材として自作した誤りの例。整列済み整数リストから一致する添字を返し、不在ならNoneという仕様。重複は任意の一致を認める。未整列入力は対象外。

1. [1] から1を探す場合を追跡し、失敗を再現する。
2. バグの原因候補と、候補を区別するテストをAIに尋ねる。
3. 修正を submission/03/search.py に作る。完成例をコピーする前に、探索範囲が閉区間か半开区間かを決める。
4. 空・1要素・末尾・不在・重複の5種類をテストする。重複例の期待値は「返した添字の値がtargetと一致する」でもよい。
5. verify.py と修正版を参照し、自分の判断を比較する。

```
python3 examples/03-ai-workflow/verify.py
python3 -m unittest discover -s submission/03 -v
```

C 相互説明

相手が選んだ入力に対する結果を、実行前に予測する。境界の更新と終了条件を説明し、実行後に一致を確認する。本人のコードの仕様を質問する活動であり、他人の提出物をAIへ送る必要はない。

提出物

search.py、test_search.py、README.md、reflection.md。記録には依頼の要点、採用/却下した候補と理由、期待値の求め方、実行結果、残っている前提を含める。提出先・期限は授業の案内に従う。

AIを使える環境がない場合は、スライドの提案例をAIの候補と見立て、相互質問で同じ検証手順を行う。環境状況と実際に行った手順を記録する。個人情報・秘密情報を依頼に含めない。

確認問題

説明の流暢さと正しさの違い、1要素の反例の意味、AIが作る期待値の検証、却下した提案の記録を、それぞれ自分の作業に結び付けて説明する。

第04回 演習：処理順と増え方

学生用。Python 3.12以上の環境で、リポジトリのルートから実行する。python3 --version で利用中の版を確認する。正式な提出先・期限は授業時の案内に従う。以下は本教材の推奨成果物。

準備

work/04 を作り、記録用の 04-observations.md を置く。

```
mkdir -p work/04
python3 examples/04-algorithms/containers.py
python3 examples/04-algorithms/trees.py
```

まず実行せずに、A、B、Cをこの順に追加したときのスタックとキューの取り出し順を予測する。木はAの子がB、C、Bの子がD、E、Cの子がF。子の列挙順を保ってDFSとBFSの訪問順を予測する。

検証の基準は、スタックがC B A、キューがA B C、木のDFSがA B D E C F、BFSがA B C D E Fとなること。実演出力と照合し、異なる場合は取り出し方法と子の順を調べる。

段階1：探索を実装する

work/04/04-search.py に、線形探索と二分探索を作る。どちらも一致する要素のインデックスを1つ返し、不在なら None を返す。二分探索の入力は昇順に整列済みであること。重複の先頭を返すことは、この段階の要件に含めない。

入力 [8, 2, 6, 4] と値6を使う。線形探索の期待値は2。二分探索には整列したコピー [2, 4, 6, 8] を渡し、期待値は2。このインデックスは整列後の列に対応する。元の列の位置に戻りたい場合は、追加の設計が必要になる。

次の条件も自分で追加する。

- 空列で不在を返す
- 先頭と末尾の要素を見つける
- 存在しない値を調べる
- 1要素の列で一致・不一致を調べる

```
python3 work/04/04-search.py
python3 examples/04-algorithms/sorting_search.py
```

参考デモのソート入力は [7, 2, 4, 2, 9]、整列後は [2, 2, 4, 7, 9]。自分の入力の結果と混同しない。

段階2：候補数を数える

3要素の列で、異なる2要素を選ぶ候補をすべて列挙する。 $i < j$ として、同じ組の重複を避ける。候補数は3。4要素なら6、5要素なら10になることを確認する。

「全探索」という名前だけで計算量を決めず、候補の数と、候補1件の確認コストを説明する。巨大な列を作らず、小さい入力で増え方を確かめる。

段階3：メモリと操作数を読む

```
python3 examples/04-algorithms/memory_complexity.py
```

出力から入力サイズと操作数モデルの値を記録する。これは実際の処理時間・比較回数を測定した値ではなく、増え方を比較する計算モデルである。入力を2倍にしたとき、線形の操作数はおよそ2倍、二重ループの操作数は4倍という関係を説明する。デモが用いるサイズ・操作の定義をコードで確認する。

メモリのbyte数には、Pythonの版・処理系・環境による差がある。`sys.getsizeof` が直接測るオブジェクトと、その参照先を区別する。全データのbyte数と断定しない。

提出用の成果物

- 04-search.py：実装と検証用入力
- 04-observations.md：予測、実際の結果、探索の前提、候補数とメモリの説明

記録にはPythonの版と実行コマンドを添える。AIを使った場合は、質問の要約・採用した提案・自分で確認した入力を書く。未使用なら、その旨を書く。最後に、資料を閉じて二分探索の区間が毎回狭くなる理由を説明する。

自分で確かめる問い

1. 未整列入力に二分探索を適用する危険を、具体例で示せるか。

2. BFSで pop() を使うと処理順はどう変わるか。
3. リストの浅いコピーは何をコピーするか。
4. 実行時間1回の測定だけで、計算量を断定できるか。

発展として、重複があるときの「最初の一致位置」を返す二分探索を考えてよい。必須範囲を検証した後に進む。

第05回 演習：グラフの設計と追跡

学生用。Python 3.12以上を使う。正式な提出方法は授業時の案内に従う。以下は本教材の推奨手順。

準備と実演

```
mkdir -p work/05
python3 examples/05-graphs/graphs.py
```

参考デモはA-B、A-C、B-D、C-Dを持つ無向グラフと、孤立頂点X。AからDのBFS経路は['A', 'B', 'D']。連結成分は2つ。出力と図を照合する。

段階1：架空校内マップを作る

次の自作の場所をグラフにする。入口A、受付B、図書室C、演習室D、別棟X。直接行き来できる通路はA-B、A-C、B-D、C-D。Xには通路がない。通路は両方向に通れ、各通路を移動1回として数える。

work/05/05-map.py に、全頂点をキーに持つ隣接リストを書く。Xの値は空のリスト。接続を出力し、図と照合する。無向辺は両方の頂点の隣接リストへ記録する。

期待する性質は、Aの隣がBとC、Dの隣がBとC、Xの隣が空であること。無向グラフの次数は、単純グラフであれば隣接頂点の数。Aの次数は2、Xは0。

段階2：BFSを手で追う

AからDを探す。Aの隣接順はB、Cとする。頂点をキューへ入れた時点で発見済みにする。開始時点を含め、取り出し1回ごとに次を記録する。

- 取り出した頂点
- 処理後に残るキュー
- 新しく追加した親の記録

Aを取り出した直後のキューはB、C。Bを処理した直後はC、D。Cを処理しても、Dを再追加しない。Dの親はBのまま。この状態からA B Dを復元できるか確認する。

段階3：入力を変える

マップのコピーでB-Dを両方向から削除する。AからDへの最少辺数は2のままだが、経路はA C Dになる。さらにC-Dも両方向から削ると、DはAから到達不能になる。元の入力に戻せるよう、変更前後のデータを分けて保存する。

到達可能な頂点を集合として出力する場合、元のグラフでAからの集合はA、B、C、D。XからはXだけ。

段階4：有向グラフへの変更

一方通行の仮想通路として、AからB、BからDだけを持つデータを別に作る。AからDには到達できるが、DからAには到達できないことを確認する。

Aの出次数は1、入次数は0。Bの出次数・入次数は両方1。次数の名前を正負の値と混同しない。

移動時間を扱う重み付きグラフへ変えるなら、BFSが最短時間を保証できる条件を 05-design.md に書く。重み付き最短経路の実装はこの回の必須範囲ではない。

提出用の成果物と検証

- 05-map.py：無向・有向の自作データと出力
- 05-design.md：頂点・辺・向き・距離の意味、BFS追跡、変更後の期待値と結果

```
python3 work/05/05-map.py
```

AIを使った範囲と、自分で照合した辺・親の記録を書く。未使用なら未使用と書く。次回に向け、キューと親辞書の役割を説明する文を残す。

自分で確かめる問い

1. 無向辺を片側だけ登録すると、どんな問題が起こるか。
2. Aから到達できる集合だけで、有向グラフ全体の強連結性を判断できるか。
3. BFSで複数の最短経路のうち1本を返す場合、何が返す経路を決めるか。
4. 孤立頂点を入力に明示する理由は何か。

第06回 演習：最少移動数の経路を実装する

学生用課題。データ構造とアルゴリズムの知識を確認し、授業内で実装・提出を行う。

Python 3.12以上。正式な提出先・締切・参照条件は授業時の案内に従う。以下の成果物構成は本教材の推奨案であり、正式な点数配分は定めない。

課題の仕様

架空駅の無向・重みなしグラフを使う。駅はA、B、C、D、E、X。辺はA-B、B-D、A-C、C-E、E-D。Xは孤立駅。

`find_route(graph, start, goal)` は、最少辺数の経路を頂点名のリストで1本返す。既知の駅同士が到達不能なら `None`。未知の駅名、または無向辺として不整合なデータには `ValueError`。既存の入力検証を保ち、仕様の条件を壊さないこと。

同じ駅を指定した場合は `[start]`。辺数は `len(path) - 1`。複数の最短経路がある入力では、1本の妥当な最短経路を返せばよい。このデータのAからDの最短経路はA B D。

CLIは出発駅と到着駅を引数に取る。省略時はA D。成功は終了コード0。到達不能と未知の駅名は終了コード2で、理由がわかるメッセージを表示する。既存のCLIを利用してよい。

準備

リポジトリのルートから実行する。

```
python3 --version
mkdir -p work/06
cp examples/06-route-project/route_starter.py work/06/route.py
cp examples/06-route-project/route_data.py work/06/route_data.py
python3 work/06/route.py A A
python3 work/06/route.py A B
python3 work/06/route.py A D
```

学生版は同駅と隣駅のみを処理する。A Dは多段経路が未実装なので失敗する。この時点の出力を記録し、実装すべき範囲を確認する。`work/06/route.py` の `find_route` を編集する。

段階1：設計を先に書く

README.md に、最短の定義、入力的前提、戻り値、終了条件を書く。キュー、親辞書、訪問済み判定の役割を説明する。未知の駅と孤立駅Xを区別する。

段階2：BFSを実装する

出発駅をFIFOのキューに入れ、親辞書に start: None を記録する。取り出した頂点の隣を調べ、未発見なら親を記録してキューに入れる。到達したgoalから親をたどり、経路を反転して返す。

同じ頂点を重複してキューに入れないこと。到達不能のgoalについて、親の復元を始めないこと。実装の各段階で、小さい入力を使って状態を追う。

段階3：必須ケースを検証する

入力	関数の期待値	CLIで確認すること
A D	['A', 'B', 'D']	移動数2、正常終了
D A	['D', 'B', 'A']	移動数2、正常終了
A A	['A']	移動数0、正常終了
A X	None	到達不能、終了コード2
UNKNOWN D	ValueError	入力エラー、終了コード2

隣駅A Bも確認する。自作の小さい入力を1件追加し、手で求めた最短経路と比較する。

```
python3 work/06/route.py A D
python3 work/06/route.py D A
python3 work/06/route.py A A
python3 work/06/route.py A X
python3 work/06/route.py UNKNOWN D
```

CLIのエラー後に終了コードを確かめる場合、直後に echo \$? を実行する。エラー表示は、この2ケースでは想定する結果である。

段階4：関数として確かめる

work/06/check_route.py を作る。単純なassertでもよいので、CLI表示と関数の結果を分けて確認する。

```
from route import find_route
from route_data import GRAPH

assert find_route(GRAPH, "A", "D") == ["A", "B", "D"]
assert find_route(GRAPH, "A", "A") == ["A"]
assert find_route(GRAPH, "A", "X") is None
path = find_route(GRAPH, "D", "A")
assert path[0] == "D" and path[-1] == "A"
assert all(b in GRAPH[a] for a, b in zip(path, path[1:]))
print("基本ケースを確認")
```

```
python3 work/06/check_route.py
```

期待出力は「基本ケースを確認」。未知の駅の例外検証や、自作入力の検証も自分で加える。上の確認だけですべての仕様を検証したことにはならない。

公開参考実装

examples/06-route-project/route_reference.py は解説・照合用の公開参考実装。先に自分の設計と結果を残す。参照した場合は範囲と学んだ内容を記録し、授業時に案内する参照条件に従う。参考版を自分の実装と偽って提出しない。

提出前の確認と成果物

work/06 に次をまとめ、案内された提出方法で授業内に提出する。提出を完了したことも確認する。

- route.py、route_data.py、check_route.py
- README.md：実行方法、Python版、仕様、設計
- test-results.md：入力、期待値、実際の結果、終了コード
- reflection.md：AIの支援範囲、提案の変更、自分で検証した内容、残る疑問

未完成なら、実行できる範囲と問題の再現入力を明示する。第07回に向け、自分のキューと親辞書がA Dでどう変化するかを説明できる状態にする。

第07回 演習：講評と質疑

学生用。共通のレビューには本教材が自作した架空提出例を使う。実学生の提出物・評価・個人情報を含めない。自分の第06回の成果物を用意する。正式な再提出の扱いは授業時の案内に従う。

段階1：架空提出例を実行する

この回のグラフは第06回のデータと異なる。無向・重みなしの辺はA-B、B-E、E-F、F-D、A-C、C-D。孤立頂点Xを含む。

```
mkdir -p work/07
python3 examples/07-review/flawed_route.py A D
python3 examples/07-review/improved_route.py A D
```

架空提出例は訪問済み管理したDFS。最初に得た経路A B E F Dを「最短」と表示する。移動数4。改善版はBFSでA C Dを返し、移動数2。この違いを図と出力で照合する。

work/07/07-review.md に、満たしている仕様、満たしていない仕様、反例、改善後に確認する条件を書く。経路が実在する点まで誤りと決め付けない。

段階2：テストを追加する

同駅A A、孤立駅A X、未知駅UNKNOWN Dを両方のCLIで試す。成功時の経路・辺数と、失敗時のメッセージ・終了コードを記録する。XからXは同駅なので経路X、移動数0となることも確認する。

発展として、作業コピーのグラフからA-Cの辺を両方向に削り、最少経路がA B E F D、4辺になるか検証する。その結果でも、元のDFSが一般に最短を保証するとは言えない理由を書く。共有の参考ファイルは変更しない。

段階3：改善の根拠を書く

「dequeを使ったから」だけで終えず、FIFOで近い層が先に処理され、未発見の頂点の親を最初の発見時に記録することを説明する。重みが異なるグラフなら、最少辺数と最少時間が一致するとは限らない。

講評文には、対象の入力、実際の結果、期待する振る舞い、改善後の確認方法を含める。能力や人格への評価を避け、コードの振る舞いを扱う。

段階4：自分の提出物を説明する

第06回のコードを開き、次の問いへ、該当箇所と具体的な入力を使って答える。

1. この関数が返す「最短」は、何を最少にした経路か。
2. キューと親辞書は、A Dの探索中にどう変化するか。
3. 同じ頂点がキューへ何度も入らない根拠はどこか。
4. 到達不能と未知の駅名をどう区別したか。
5. 自作テストは、どの実装ミスを検出する狙いで選んだか。
6. AIの提案を採用した箇所について、自分が確認したことは何か。

担当教員との授業内の質疑で説明する。説明に詰まった場合は、入力を小さくして状態を書き出す。質疑で気付いた疑問・修正点を残す。

段階5：改善を検証して記録する

work/07/07-improvements.md に、第06回の修正前後の振る舞い、再現入力、期待値、実際の結果を書く。修正が不要なら、確認した条件とその証拠を記す。AIを使用したか、何に使い、何を自分で検証したかを振り返る。

成果物の推奨構成は 07-review.md と 07-improvements.md。実装も修正した場合は、修正版コードとテスト結果を一緒に整理する。提出・再提出するかは授業時の案内に従う。

次回への準備

電卓の入力文字列をどう区切るかを考える。例は $12 + 3 * 4$ 。数と演算子を識別し、演算のまとまりを木で表す場面を予想する。次回は字句解析・構文解析・意味解析の区別を学ぶ。

第8回 電卓の解析と実行

到達目標

字句・構文・意味の境界を、トークン、AST、テストで説明する。Python 3.12以上、木の走査と関数を復習する。公開referenceはexamples/08-calculator/calculator.py、動作する拡張用starterはstarter.py。Pythonのeval/execを使わない。

A トークンの追跡

$12+3.5*(2-1)$ を種類・文字列・開始位置に分け、末尾eofも記録する。空白を挿入した場合、意味は同じで開始位置が変わることを確かめる。 $2@3$ を受理せず、@の位置を説明する。

B ASTと評価

$2+3*4$ 、 $(2+3)*4$ 、 $10-3-2$ の木を描く。期待値は14、20、5。子の評価が終わるごとに、途中結果を書き込む。

C 実装または拡張

submission/08に電卓を作り、字句・構文・評価を別関数に分ける。参照版はスライドのEBNF全体に対応する。学生演習の基本必須は整数表記と 3.5 のように小数点の前後に数字がある表記。3.、.5 は発展として実装してもよい。対応範囲をREADMEに明示し、基本例は全て満たす。

starter.py は基礎の電卓を呼ぶ動作例。拡張課題は、独立の検査・表示・テストを増やすこと。自力実装する場合も完成例と比較してよい。

受入例：

入力	期待値または失敗
2+3*4	14
(2+3)*4	20
-3*2	-6
8/2/2	2
0.1+0.2	誤差を考慮して0.3に近い
2@3	字句エラー
(2+3	構文エラー
2 3	余分なトークンとして構文エラー
1/0	評価エラー

```
python3 examples/08-calculator/calculator.py --eval '2+3*4'
python3 -m unittest discover -s submission/08 -v
```

エラーメッセージを出しただけで合格にせず、該当段階をテストする。例を手計算してから実行する。

提出物

実装、テスト、README（文法と実行方法）、AST図、reflection（AI採用判断、失敗と修正、段階の説明）をまとめる。提出先・期限は授業の案内に従う。

確認

2+*3は読めるトークンだが並びが不正。ASTは空白そのものより意味を持つ構造を残す。10-3-2は左結合。ゼロ除算は構文として合法的な式の評価条件である。自分の処理系で各理由を説明する。

第9回演習 LispのREPLと四則演算

教材と作業用コード

PythonホストとLisp方言の仕様は本教材の設計上の選択であり、正式シラバスの指定言語仕様ではない。

examples/09-lisp-arithmetic/lisp.py は実行可能な完成参照。starter.py は既存機能を使いながら 既存の square 組込みへ入力検証を追加するウォームアップ用の実行可能 starterで、講義の全機能が未実装の穴埋めではない。ウォームアップでは作業用に複製した starter.py の machine.add_builtin にあるlambdaを名前付き関数へ置き換え、数値検証の後で負数なら LispError を送出する。正常入力と負数の反例を確かめる。square は meta側へ自動登録されない。

本課題は前回の自作版、または別名の作業用コードで進める。完成参照を使った範囲を明記し、自分の実装と混同しない。比較のために参照版をそのまま残す。

演習手順と成果物は教材案。公式の提出先、締切、点数配分をここでは規定しない。

到達目標

S式の読取と四則演算の評価を分け、失敗後も次の入力を受けるREPLを作る。

始める前に

第8回のトークン、木、再帰の説明を読み直す。Python 3.12以上を使い、プロジェクトのルートから実行する。第9回のホストREPLは1行1式で扱う。複数行の式入力はこの段階の受入条件に含めない。

段階1 読取

(+ 1 (* 2 3)) のトークン列と木を手で書く。読取関数へ渡し、数値・記号・入れ子リストの区別を確認する。閉じ括弧不足と余分な閉じ括弧をそれぞれ入力し、評価へ進む前に失敗することを確認する。

段階2 評価

前回の自分のコードまたは新しい作業用ファイルに、数値と二項の +、-、*、/ を実装する。通常の手出しは引数を評価してから計算する。単項演算や可変長引数を追加する場合

は、対応仕様を明記する。完成参照の + と * は0個以上、- と / は1個以上。単項減算は符号反転、単項除算は逆数。複数引数の減算と除算は左から順に計算する。まず二項の版を通し、参照の契約へ段階的に広げる。

段階3 REPL

入力、読取、評価、表示を循環する。1式の失敗を報告した後、次の式を受ける。:quit と EOFを終了として扱う。診断文の完全一致より、失敗原因を識別でき、循環が継続することを確認する。

受入例

入力	期待
(+ 2 3)	5
(- 9 4)、(* 3 4)	5、12
(/ 8 2)	数値として 4
(* (+ 2 3) (- 9 4))	25

失敗例は (/ 1 0)、(+ 1 2、(+ 1 2))、(unknown 1 2)。失敗後に (* 4 5) を入力して 20 を得る。読み取りと評価のどちらで失敗したか説明する。

検証コマンド

```
python3 examples/09-lisp-arithmetic/lisp.py --eval "(+ 1 (* 2 3))"
python3 examples/09-lisp-arithmetic/lisp.py
```

参照版の結果は 7。2行目のREPLで正常、エラー、正常の順に入力する。自分の版は同じ入力と比較する。この回はファイル実行を課題にせず、第11回で追加する。

成果物案

- 自分の読取・評価・REPLのコードと実行コマンド
- トークン列と木を示す説明
- 受入例、失敗例、回復例のログ

- 対応する数値・引数個数・表示の仕様

AI利用の振り返り

AIに求めた説明または修正案、採用した理由、自分で実行した反例を記録する。生成されたコードを説明できない箇所は、関数を小さく分けて読取と評価を追う。

確認問い

(+ 1 2) で + を先頭に置く意味は何か。括弧不足とゼロ除算を同じ診断にすると、誰が何を困るか。

第10回演習 変数と関数の定義

教材と作業用コード

PythonホストとLisp方言の仕様は本教材の設計上の選択であり、正式シラバスの指定言語仕様ではない。

`examples/10-lisp-functions/lisp.py` は実行可能な完成参照。`starter.py` は既存機能を使いながら 既存の `square` 組込みへ入力検証を追加するウォームアップ用の実行可能starterで、講義の全機能が未実装の穴埋めではない。ウォームアップでは作業用に複製した `starter.py` の `machine.add_builtin` にある `lambda` を名前付き関数へ置き換え、数値検証の後で負数なら `LispError` を送出する。正常入力と負数の反例を確かめる。`square` は meta側へ自動登録されない。

本課題は前回の自作版、または別名の作業用コードで進める。完成参照を使った範囲を明記し、自分の実装と混同しない。比較のために参照版をそのまま残す。

演習手順と成果物は教材案。公式の提出先、締切、点数配分をここでは規定しない。

到達目標

環境による名前検索、`define`、`lambda`、`if`、`quote` を実装し、クロージャと再帰を説明する。

始める前に

第9回の自分の版を作業用に複製する。数値・記号・リストを区別できることを確認する。文字列やファイル実行は第11回の範囲。複数の式を連続実行する参照検証には `begin` を使える。

段階1 環境

評価関数に環境を渡す。記号は現在の環境から外側へ探し、見つからなければ診断する。`define` は名前の式を評価せず、右辺だけを評価して登録する。定義の返り値は自分の仕様書き、定義後の名前の値を受入条件にする。

段階2 関数

lambda は仮引数、本文、定義時環境を保存する値を作る。呼出し時には実引数を評価し、新しい環境へ束縛する。新しい環境の外側は関数の保存環境とする。引数個数が合わない場合を、束縛前に診断する。

段階3 特殊形式

if は条件と選択した枝だけを評価する。教材方言では #f が偽で、数値 0 や空リストも真として扱う。quote は受け取った式を評価せず返す。通常関数適用とは別の分岐にする。

段階4 再帰とクロージャ

```
(define make-adder
  (lambda (x) (lambda (y) (+ x y))))
(define add5 (make-adder 5))
(add5 3)
```

上の改行は説明用。教材REPLは1行ごとに1式を読むため、各 define のまとまりを1行にして同じセッションへ入力し、8を確認する。次のCLIなら改行を含む定義を1個の begin として検証できる。

```
python3 examples/10-lisp-functions/lisp.py --eval '
(begin
  (define make-adder
    (lambda (x) (lambda (y) (+ x y))))
  (define add5 (make-adder 5))
  (add5 3))'
```

外側の関数が戻った後の環境を図にする。

```
(define fact
  (lambda (n)
    (if (= n 0) 1
        (* n (fact (- n 1))))))
(fact 4)
```

こちらの define fact も、REPLには定義全体を1行にして入力する。fact 0 は 1、fact 4 は 24。負数など未対応の入力を、検証済みの範囲と区別する。

受入例

入力	期待
<code>(begin (define x (+ 2 3)) x)</code>	5
<code>(begin (define square (lambda (x) (* x x))) (square 6))</code>	36
<code>(if #f (/ 1 0) 7)</code>	7、ゼロ除算なし
<code>(quote (+ 1 2))</code>	リスト (+ 1 2)

追加の反例として未定義名、square の引数個数違いを記録する。外側の `x = 100` と引数 `x = 4` を共存させ、呼出し後も外側が 100 のままであることを確認する。

検証コマンド

```
python3 examples/10-lisp-functions/lisp.py --eval '(if #f (/ 1 0) 7)'  
python3 examples/10-lisp-functions/lisp.py
```

成果物案

実装コード、環境図、受入例と反例のログ、特殊形式の評価順序の説明を残す。定義時と呼出し時の環境を別々に描く。

AI利用の振り返り

AIに環境探索を説明させ、自分の図と比較する。実行前の予測、反例での観察、修正した説明を保存する。AIが動的のスコープを提案した場合は `make-adder` などで検討する。

確認問い

`lambda` の本文を定義時に評価すると何が起きるか。`if` に両方の枝の値を先に渡してはならない理由は何か。

第11回演習 スクリプト実行と評価器の準備

教材と作業用コード

PythonホストとLisp方言の仕様は本教材の設計上の選択であり、正式シラバスの指定言語仕様ではない。

examples/11-lisp-scripts/lisp.py は実行可能な完成参照。starter.py は既存機能を使いながら 既存の square 組込みへ入力検証を追加するウォームアップ用の実行可能starterで、講義の全機能が未実装の穴埋めではない。ウォームアップでは作業用に複製した starter.py の machine.add_builtin にあるlambdaを名前付き関数へ置き換え、数値検証の後で負数なら LispError を送出する。正常入力と負数の反例を確かめる。square は meta側へ自動登録されない。

本課題は前回の自作版、または別名の作業用コードで進める。完成参照を使った範囲を明記し、自分の実装と混同しない。比較のために参照版をそのまま残す。

演習手順と成果物は教材案。公式の提出先、締切、点数配分をここでは規定しない。

到達目標

UTF-8のファイルから複数式を順に実行し、Lisp製評価器に必要なリスト・型・入出力操作を確かめる。

段階1 スクリプト入口

第10回の自分の版にファイル実行の入口を加える。ファイル全体をUTF-8で読み、終端まで式を繰り返し読む。同じ環境で順に評価する。ファイルを開けない失敗、読取時の失敗、評価時の失敗を区別する。スクリプトでの失敗後の方針はREADMEに記述する。参照版は実行を停止して非ゼロ終了する。

作業用 lesson11.lisp に次を保存する。

```
(define square (lambda (x) (* x x)))
(display (square 5))
(newline)
(display (square 7))
(newline)
```

表示は 25 と 49。参照版のCLIは最後の式の値だけを自動表示する。この例では末尾の `newline` の返り値 () も最後に表示する。途中の値を表示するには `display` と `newline` を使う。

段階2 字句の状態

"hello world"、"a;b"、行コメント ; memo を試す。文字列中の空白と ; を失わないことを確かめる。記号と文字列を内部の型で区別する。括弧を文字列内に含めた例も追加する。

段階3 リスト操作

`list`、`car`、`cdr`、`cons`、`null?` を使い、空・1要素・複数要素のリストを調べる。空リストから先頭を取る失敗も記録する。

```
(define total
  (lambda (xs)
    (if (null? xs) 0
        (+ (car xs) (total (cdr xs))))))
(display (total (quote (2 3 4))))
(newline)
```

結果は 9。total に空リストを渡すと 0。

段階4 セルフホスティングの準備

Lispの関数へ (quote (+ 1 2)) を渡し、car と cdr で式を分解する。数値・記号・リストを判定する操作を試す。次回の評価器で必要になる操作を、対象の式と対応付ける。読取やOS入出力をどの層に任せるかも記述する。

受入例

入力・状況	期待
上の lesson11.lisp	25、49、最後の値 () を改行して表示
(car (quote (10 20)))	10
(cdr (quote (10 20)))	(20)
(cons 5 (quote (10 20)))	(5 10 20)

加えて、存在しないファイル、括弧不足のファイル、未定義名のファイルを検証する。失敗の段階を説明し、終了状態も記録する。

検証コマンド

```
python3 examples/11-lisp-scripts/lisp.py lesson11.lisp  
python3 examples/11-lisp-scripts/lisp.py --eval '(null? (quote ()))'
```

2行目の結果は #t。作業用ファイルを保存した場所から実行し、別の場所で行う場合はパスを合わせる。

成果物案

自分の実装、作業用スクリプト、実行コマンド、正常・失敗ログ、評価器に必要な操作の一覧を残す。新しい操作を追加する場合は、入力と期待する型も記述する。

AI利用の振り返り

AIの提案する文字列の字句解析を、空白・コメント記号・括弧の反例で確かめる。採用したコードと、検証していない入力を区別して記録する。

確認問い

ファイルを空白だけで分割すると、どの入力が壊れるか。式ごとに環境を初期化すると、スクリプトの何ができなくなるか。

第12回演習 Lisp製評価器の最小実装

教材と作業用コード

PythonホストとLisp方言の仕様は本教材の設計上の選択であり、正式シラバスの指定言語仕様ではない。

`examples/12-meta-evaluator/lisp.py` は実行可能な完成参照。`starter.py` は既存機能を使いながら 既存の `square` 組込みへ入力検証を追加するウォームアップ用の実行可能starterで、講義の全機能が未実装の穴埋めではない。ウォームアップでは作業用に複製した `starter.py` の `machine.add_builtin` にある`lambda`を名前付き関数へ置き換え、数値検証の後で負数なら `LispError` を送出する。正常入力と負数の反例を確かめる。`square` は meta側へ自動登録されない。

本課題は前回の自作版、または別名の作業用コードで進める。完成参照を使った範囲を明記し、自分の実装と混同しない。比較のために参照版をそのまま残す。

`evaluator_starter.lisp` は自己評価される数値・真偽値・文字列だけを扱う実行可能な最小starter。`starter-eval 42` の値を確認してから、引用と算術を段階的に追加する。`evaluator.lisp` は完成参照。第14回以降の `repl.lisp` も完成参照である。

演習手順と成果物は教材案。公式の提出先、締切、点数配分をここでは規定しない。

到達目標

ホストと対象言語を分け、Lispで書いた最小評価器を動かす。必要な機能の契約を、具体的な対象式で裏付ける。

始める前に

第11回の自分の版を使うか、完成ホスト `lisp.py` を基盤としてLisp製評価器を自作する。ホストの不足機能を補う課題と、対象の評価規則を書く課題を混ぜない。基盤として完成ホストを使った場合は、そのことを成果物に明記する。

段階1 機能契約

数値判定、記号判定、リスト分解、環境探索の操作について、入力、結果、失敗条件を記述する。どの対象式を処理するために必要かを添える。Pythonホストの変数と、対象環境の束縛を図で分ける。

段階2 最小評価器

作業用 `my-evaluator.lisp` に、数値をそのまま返す処理を作る。次に `quote` と二項の四則演算を追加する。入力は読取済みS式にする。対象の式を渡すときには、ホストによる先行評価を `quote` で止める。

```
(quote (+ 1 (* 2 3)))
(quote (quote (+ 1 2)))
```

上は対象の計算式をデータとして渡す例。下は対象で引用を処理するためのデータ。2個の `quote` がそれぞれどの層で外れるか説明する。

段階3 分岐

対象の `if` を追加する。対象条件を評価してから片方の枝だけを対象評価器で評価する。
(`if #f (/ 1 0) 7`) が 7 を返すことを確認する。未対応の対象構文を、ホストへ黙って丸投げせず診断する。

段階4 次の機能の計画

名前検索と対象の `define`、対象のクロージャへ拡張する契約を設計する。第12回の完成参照のmeta評価器は数値・四則・`quote`・`if`などの最小段階である。対象の関数や環境の完成は第13～14回の参照へ進む。ホスト側に `lambda` があることと、対象側に実装できたことを区別する。

受入例

対象の入力	期待
7	7
(+ 1 (* 2 3))	7
(quote (+ 1 2))	(+ 1 2)
(if #f (/ 1 0) 7)	7、ゼロ除算なし

反例は未知の名前、未知の演算子、引数型の違い。対象側でどこまで診断できるかを書き、未対応機能も明記する。

完成参照の検証

```
python3 examples/12-meta-evaluator/lisp.py --meta --eval '(+ 1 (* 2 3))'
python3 examples/12-meta-evaluator/lisp.py --meta --eval '(if #f (/ 1 0) 7)'
```

どちらも 7。--meta は同じディレクトリの evaluator.lisp を読み、その m-eval を Python製Lispホスト上で動かす。自作版を試すときは、完成参照のファイルを上書きせず、作業用スクリプトとしてホストで実行する。

成果物案

Lisp製最小評価器、必要な操作の契約、ホストと対象を分けた図、受入例と反例のログ、未対応機能と次の開発順序を残す。

AI利用の振り返り

AIの案が対象の名前をホスト環境で検索していないか確認する。提案、確認入力、採用理由または却下理由を記録する。

確認問い

ホストに式を評価させるだけの関数と、対象評価器の実装はどう異なるか。組込みへ委譲した範囲をどの証拠で示せるか。

第13回演習 理解確認とディスカッション

教材と作業用コード

PythonホストとLisp方言の仕様は本教材の設計上の選択であり、正式シラバスの指定言語仕様ではない。

`examples/13-language-review/lisp.py` は実行可能な完成参照。`starter.py` は既存機能を使いながら 既存の `square` 組込みへ入力検証を追加するウォームアップ用の実行可能 `starter` で、講義の全機能が未実装の穴埋めではない。ウォームアップでは作業用に複製した `starter.py` の `machine.add_builtin` にある `lambda` を名前付き関数へ置き換え、数値検証の後で負数なら `LispError` を送出する。正常入力と負数の反例を確かめる。`square` は `meta` 側へ自動登録されない。

本課題は前回の自作版、または別名の作業用コードで進める。完成参照を使った範囲を明記し、自分の実装と混同しない。比較のために参照版をそのまま残す。

`evaluator_starter.lisp` は自己評価される数値・真偽値・文字列だけを扱う実行可能な最小 `starter`。`starter-eval 42` の値を確認してから、引用と算術を段階的に追加する。`evaluator.lisp` は完成参照。第14回以降の `repl.lisp` も完成参照である。

演習手順と成果物は教材案。公式の提出先、締切、点数配分をここでは規定しない。

到達目標

コードを実行する前に結果を予測し、環境と評価順序を説明する。他者の説明を反例で確かめ、自分の説明を更新する。

今日の進め方

新機能の一斉実装を主活動にしない。個人予測、ペア説明、実行、反証、説明の修正を繰り返す。自分の成功例と未解決例を持ち寄り、互いのコードを尊重して質問する。

議題1 引用と分岐

`(quote missing)`、`(if #f (/ 1 0) 8)`、`(if (= 1 1) 9 missing)` の結果を先に書く。次に、評価しない部分へ印を付ける。相手は「通常の関数適用で同じ実装ができるか」と追質問する。実行後、最初の説明との差を記録する。

議題2 クロージャ

```
(begin
  (define make-adder
    (lambda (x) (lambda (y) (+ x y))))
  (define plus2 (make-adder 2))
  (plus2 7))
```

期待する値は 9。x の束縛がどこに残るか図にする。相手は別の加算器を作って、保存した環境が混ざらないか確かめる。動的スコープという説明を受けたら、どの入力で差が出るか検討する。

議題3 環境の分離

外側の x = 100 と関数の引数 x = 4 を用意する。関数の結果 5 と、呼出し後の外側の値 100 を予測する。単一の変数表を使う架空の案を挙げ、反例で検討する。

議題4 追跡

```
python3 examples/13-language-review/lisp.py --trace --eval '(+ 1 (* 2 3))'
python3 examples/13-language-review/lisp.py --trace --meta --eval '(+ 1 (* 2
3))'
```

結果は両方 7。追跡は標準エラーへ出力する。[host depth=N] はホスト、[meta depth=N] はLisp製評価器の追跡。深さと自分の木を照合する。meta実行中にもホストの処理がある理由を説明する。ログだけでは分からない環境情報がある場合は、追加する証拠を提案する。

議題5 達成範囲

Lisp製評価器が担う処理と、Pythonホストへ委譲する読取・入出力を図にする。対象の quote、if、関数適用がどの層で動くか、関連コードを示す。ネイティブ起動や独立ブートストラップを達成したとは主張しない。

相互質問

相手へ、理解した箇所の問い1個と追質問1個を出す。相手の答えを正誤だけで処理せず、反例や追跡で確かめる。自分が説明できなかった点は、次の開発課題にする。

受入条件

記録	必要な内容
予測	実行前の値と理由
観察	入力・結果・コマンド
説明	環境図または評価の順序
更新	反例と修正後の説明

未知の小さな式について1件、上記がそろふことを目指す。これは授業用の到達確認であり、点数表ではない。

成果物案

議論記録、相手へ出した問い、環境図、追跡ログ、未解決の問いと次に試す入力を残す。相手の個人情報や提出物を公開せず、自分の記録としてまとめる。

AI利用の振り返り

AIによる実装説明から、検証が必要な主張を1個選ぶ。自分の反例やログで確かめ、説明をどう修正したか書く。AIの説明をそのまま相手への回答にしない。

第14回演習 Lisp製評価器とREPL

教材と作業用コード

PythonホストとLisp方言の仕様は本教材の設計上の選択であり、正式シラバスの指定言語仕様ではない。

examples/14-selfhosting/lisp.py は実行可能な完成参照。starter.py は既存機能を使いながら 既存の square 組込みへ入力検証を追加するウォームアップ用の実行可能starterで、講義の全機能が未実装の穴埋めではない。ウォームアップでは作業用に複製した starter.py の machine.add_builtin にあるlambdaを名前付き関数へ置き換え、数値検証の後で負数なら LispError を送出する。正常入力と負数の反例を確かめる。square は meta側へ自動登録されない。

本課題は前回の自作版、または別名の作業用コードで進める。完成参照を使った範囲を明記し、自分の実装と混同しない。比較のために参照版をそのまま残す。

evaluator_starter.lisp は自己評価される数値・真偽値・文字列だけを扱う実行可能な最小starter。starter-eval 42 の値を確認してから、引用と算術を段階的に追加する。evaluator.lisp は完成参照。第14回以降の repl.lisp も完成参照である。

演習手順と成果物は教材案。公式の提出先、締切、点数配分をここでは規定しない。

到達目標

Lispで書いた対象評価器に、名前、クロージャ、再帰を統合し、Lisp製REPLをPythonホスト上で実行する。継続する環境と、委譲した読取・入出力の範囲を説明する。

始める前に

第12回の自作評価器と第13回の未解決記録を用意する。完成参照 evaluator.lisp と repl.lisp は比較用。自分のファイルで開発し、参照版を編集しない。完成ホストを使う場合、その基盤を明記する。

段階1 対象評価器

数値、記号、リストの分類を確認する。define の右辺と、if の条件・選択枝を対象評価器で処理する。対象の関数は仮引数・本文・定義時環境を保存する。通常の適用で引数を評価してから環境を拡張し、本文を評価する。

段階2 クロージャと再帰

対象の `square`、`make-adder`、`fact` を実行する。対象の環境を図にして、呼出しごとの束縛と共有する外側を分ける。再帰では関数自身の定義を環境で見つける必要がある。

```
(define make-adder
  (lambda (x) (lambda (y) (+ x y))))
(define add5 (make-adder 5))
(add5 3)
```

Lisp製REPLへ順に入力すると最後の値は 8。

段階3 Lisp製REPL

Lisp側で読取、対象評価、表示の循環を組み立てる。読取済みS式を対象評価器へ渡し、対象環境を次の入力へ引き継ぐ。読取とOS入出力はPythonホストの組込みへ委譲する。エラーの保護機能もホストを使う。対象評価を、ホストの通常評価で置き換えない。

段階4 継続と終了

`define`、正常式、エラー式、正常式の順に入力し、定義が残り、次の入力へ復帰することを確かめる。EOFと `:quit` で終了する。教材REPLは1行ごとに読取る。Lisp製REPLは同じ行の複数式も扱えるが、複数行補完はしないので関数定義は1行にして入力する。

受入例

対象の入力	期待
(begin (define square (lambda (x) (* x x))) (square 6))	36
上の make-adder の例	8
(begin (define fact (lambda (n) (if (= n 0) 1 (* n (fact (- n 1)))))) (fact 4))	24
define x=5、(+ x 2)、(/ 1 0)、(+ x 3)	7、診断、 8

最後の行の定義は実際には `(define x 5)` と入力する。反例に未定義名、引数個数違い、非選択枝の失敗を含める。

完成参照の検証

```
python3 examples/14-selfhosting/lisp.py --meta --eval '(+ 1 (* 2 3))'  
python3 examples/14-selfhosting/lisp.py --selfhost-repl
```

1行目はLisp製評価器の `m-eval` を実行して 7 を得る。2行目は `repl.lisp` によるLisp製REPLを起動する。無引数の `lisp.py` はホストのREPLであり、起動方法の違いを記録する。

成果物案

Lisp製評価器とREPL、ホスト起動コマンド、対象入力と結果のログ、環境図、委譲する操作の一覧、未対応の構文を残す。ログと関連コードで、Lisp製評価器を実際に呼んだことを示す。

達成報告は「Python製Lispホスト上で、Lisp製評価器とREPLを実行した」とする。読取器のLisp実装、OSへの直接入出力、ネイティブ実行、独立ブートストラップはこの教材の成果と区別する。

AI利用の振り返り

AIの案がホスト環境と対象環境を混同していないか確かめる。第13回の反例を回帰検証として再実行し、採用した案と結果を記録する。

確認問い

同じ結果が出るだけで、Lisp製評価器を使った証明になるか。エラー後も保持する状態と、捨てる状態は何か。

第15回演習 講評と総括

教材と作業用コード

PythonホストとLisp方言の仕様は本教材の設計上の選択であり、正式シラバスの指定言語仕様ではない。

`examples/15-showcase/lisp.py` は実行可能な完成参照。`starter.py` は既存機能を使いながら既存の `square` 組込みへ入力検証を追加するウォームアップ用の実行可能starterで、講義の全機能が未実装の穴埋めではない。ウォームアップでは作業用に複製した `starter.py` の `machine.add_builtin` にある `lambda` を名前付き関数へ置き換え、数値検証の後で負数なら `LispError` を送出する。正常入力と負数の反例を確かめる。`square` はmeta側へ自動登録されない。

本課題は前回の自作版、または別名の作業用コードで進める。完成参照を使った範囲を明記し、自分の実装と混同しない。比較のために参照版をそのまま残す。

`evaluator_starter.lisp` は自己評価される数値・真偽値・文字列だけを扱う実行可能な最小starter。`starter-eval 42` の値を確認してから、引用と算術を段階的に追加する。`evaluator.lisp` は完成参照。第14回以降の `repl.lisp` も完成参照である。

演習手順と成果物は教材案。公式の提出先、締切、点数配分をここでは規定しない。

到達目標

開発した実装の達成範囲を、仕様・コード・実行結果で説明する。他者の問いを受けて改善案を選び、授業全体を振り返る。

準備

自分の最新コード、仕様、受入例、実行ログを用意する。未完成の箇所も制限として示す。実在の他者の提出物や成績を公開しない。`examples/15-showcase/` は教材用の架空の例であり、学生の提出物ではない。

段階1 架空の例の講評

講義の例Aは算術だけの実装、例Bは環境を共用し過ぎる設計案、例CはPythonホスト上のLisp製評価器という架空の説明である。付属fixtureは受入入力材料。完成実装の動作

と、架空の不具合案の説明を区別する。例Bの誤りは、外側の $x = 100$ と関数引数 $x = 4$ が衝突する入力で検討する。

段階2 自分の実装の紹介

成功例1件について、結果を予測してから実行する。その結果を生む評価規則と環境を図にする。関連するコードの位置を示す。未対応または失敗する入力も1件示し、診断か制限を説明する。

段階3 相互講評

聞き手は作者の説明から曖昧な名前や順序を1つ選び、小さな反例を提案する。作者は結果を予測し、実行して確かめる。観察した事実と改善提案を分けて記録する。意図が不明な箇所は作者へ確認する。

段階4 改善計画

改善を1つ選び、利用者に起きる変化、変更する箇所、受入例を記述する。実装した場合は結果を記録する。計画のみの案を完成済み機能として数えない。

受入条件

成果	確認すること
達成範囲	対応構文と未対応機能を示す
実行証拠	コマンドと再現できる入力がある
講評記録	事実と改善提案を分ける
振り返り	判断が変わった具体例を示す

上記は教材の到達確認案であり、正式な採点配分ではない。

参照の検証

```
python3 examples/15-showcase/lisp.py --meta --eval '(+ 1 (* 2 3))'
python3 examples/15-showcase/lisp.py --selfhost-repl
```

算術の結果は 7。REPLでは自分の紹介例を試す。python3 examples/15-showcase/check_compatibility.py は自作fixture compatibility.json の正常・失敗ケースをホストとmetaで比較する。Lisp製評価器とホストを区別し、読取・入出力の委譲を説明する。

成果物案

自分のコード、仕様、受入例、実行ログ、講評記録、改善計画、全体の振り返りを相互に対応付ける。提出先・締切・正式採点運用は授業で確認する。

全体の振り返り

構文木と環境探索に、前半のデータ構造やアルゴリズムがどう役立ったか、コードの例で説明する。読取、評価、環境、入出力の境界を分けたことで、変更の広がりがどう変わったか振り返る。

AI利用の振り返り

AIへ任せた作業、自分が判断した部分、採用前の検証を具体的に記録する。最初の誤った説明を反例で修正した経験があれば、何を学んだか書く。検証していない主張は未確認と明記する。